

Customer Support Ticket Triage and Resolution

Technical documentation set

20 documents

Documentation set: Customer Support Ticket System (Enklima)

Version 1, 2026-07-28. Written against the code as it stands, not as planned.

#	Document	What it answers
01	Requirements	What the system must do, numbered and testable
02	High-Level Design	The components, the data flow, the tech choices
03	Low-Level Design	Inside each component: modules, nodes, state, error handling
04	Data Model	Tables, columns, the vector collection, retention
05	API Specification	Every endpoint, its inputs, outputs, and status codes
06	Decision Log (ADRs)	Each architectural decision, its options and consequences
07	Configuration and Secrets	Every environment variable and where its real value lives
08	Runbook	Start, stop, seed, recover, and what to do when it breaks
09	Test Plan and Report	What is tested, how, and the last measured result
10	Benchmark Report	Measured escalation, confidence, and latency numbers
11	Non-Functional Requirements	Performance, availability, security targets with numbers
12	Security Review	Threat model: what an attacker could try and what stops them
13	Privacy and Data Handling	What personal data is touched, where it goes, how long it stays
14	Model Card	Which models, their limits, known failure modes
15	Risk Register	What could sink the project and what mitigates it
16	Traceability Matrix	Requirement to code to test, so coverage is provable
17	User Guide	How a non-technical user operates the system
18	Release Notes	What changed, in order, and what it broke
19	Handover	Everything the next owner needs

Also in this folder: the source requirement PDF from the client, and [demo-script.md](#) for the walkthrough.

01. Requirements

Version 1, 2026-07-28. Authoritative sources: the client requirement PDF in this folder, and the project PRD ([Track-Use-Cases/01-Customer-Support-Triage/PRD.md](#) in the planning repo). This document restates those requirements in the form the code is checked against. Status is measured, not aspirational: see [16-traceability-matrix.md](#) for the evidence behind every row.

1. Problem

A support team receives tickets in mixed formats. For each one a human judges urgency, searches knowledge articles and past resolutions, drafts a reply, and confirms it follows policy before sending. That is slow, inconsistent between agents, and risky on tickets carrying private customer data.

2. What this product is

A multi-agent assistant **for a support agent**, not an end-customer chatbot. It ingests a ticket, classifies and prioritises it, checks sensitivity locally, retrieves relevant knowledge, drafts a compliant reply, then either sends low-risk high-confidence replies automatically or escalates the rest to the right human. Every step appends to a tamper-evident audit trail, and resolved tickets are indexed back as searchable precedent.

It is also the reusable skeleton for the legal-review and AI-governance systems, so clean module boundaries are a requirement rather than a preference.

3. Functional requirements

ID	Requirement	Status
FR-1	Accept tickets in multiple formats (web form, email, voice transcript as text) and normalise each into one canonical ticket schema. A documented adapter interface exists for external ticketing products; no live integration is built.	MET, except the transcript format is unreachable from the API
FR-2	Label each ticket with issue type, priority, business impact, sentiment, difficulty, and a sensitivity flag with reasons.	MET, except no classification confidence score
FR-3	Detect sensitivity locally BEFORE any third-party API call: pattern matching for PII plus a model check for contextual sensitivity.	MET
FR-4	Route the model by a 2x2 table over sensitivity and difficulty. Sensitive tickets never reach a third-party API.	MET
FR-5	Retrieve relevant knowledge-base articles and past resolutions from the vector database, ranked, with source ids and scores.	MET
FR-6	Draft a reply with the routed model, citing sources, with a confidence score.	MET, citation is free text not a structured field
FR-7	Check the draft against policy, tone and risky language. On failure regenerate once; a second failure escalates.	MET
FR-8	Decide auto-send versus escalate by plain code, and pick an assignee from the team roster on escalation.	PARTIAL, high priority and sensitivity raise the bar rather than forcing escalation
FR-9	Reviewer interface: approve, reject, or edit the draft before it goes out.	MET
FR-10	Append every step to a tamper-evident hash chain; editing any past entry is detectable.	MET

ID	Requirement	Status
FR-11	Index resolved tickets back into the vector database as retrievable precedent, with no model fine-tuning.	MET

4. Non-functional requirements

ID	Requirement	Target	Status
NFR-1	Sensitive tickets are processed only on the self-hosted lane, and never fall back to cloud when that lane is down.	Hard rule, zero exceptions	PARTIAL, no leak is possible, but an outage ends in an error status instead of a human escalation
NFR-2	Near-real-time handling, with latency visible in the interface.	Cloud lane seconds, private lane allowed slower	MET
NFR-3	Single concurrent user, synthetic corpus in the low hundreds.	Not high-concurrency	MET
NFR-4	The pipeline never crashes: a failing model call is retried and the failure is visible.	Two attempts, 180s cap	PARTIAL, no backoff and no fallback to a healthy model
NFR-5	The accumulating state object is itself the audit record.	Every node auditable	MET
NFR-6	Module boundaries clean enough that the skeleton is reusable by the other two systems.	Reuse proven	MET
NFR-7	Synthetic data only; secrets in environment variables, never in the repository.	Zero real customer data	MET

5. Out of scope

No speech to text (transcripts arrive as text). No live external ticketing integration (adapter interface only). No model fine-tuning. Not a customer-facing self-serve chatbot. Not production grade: no high availability, no high concurrency.

6. Known requirement gaps

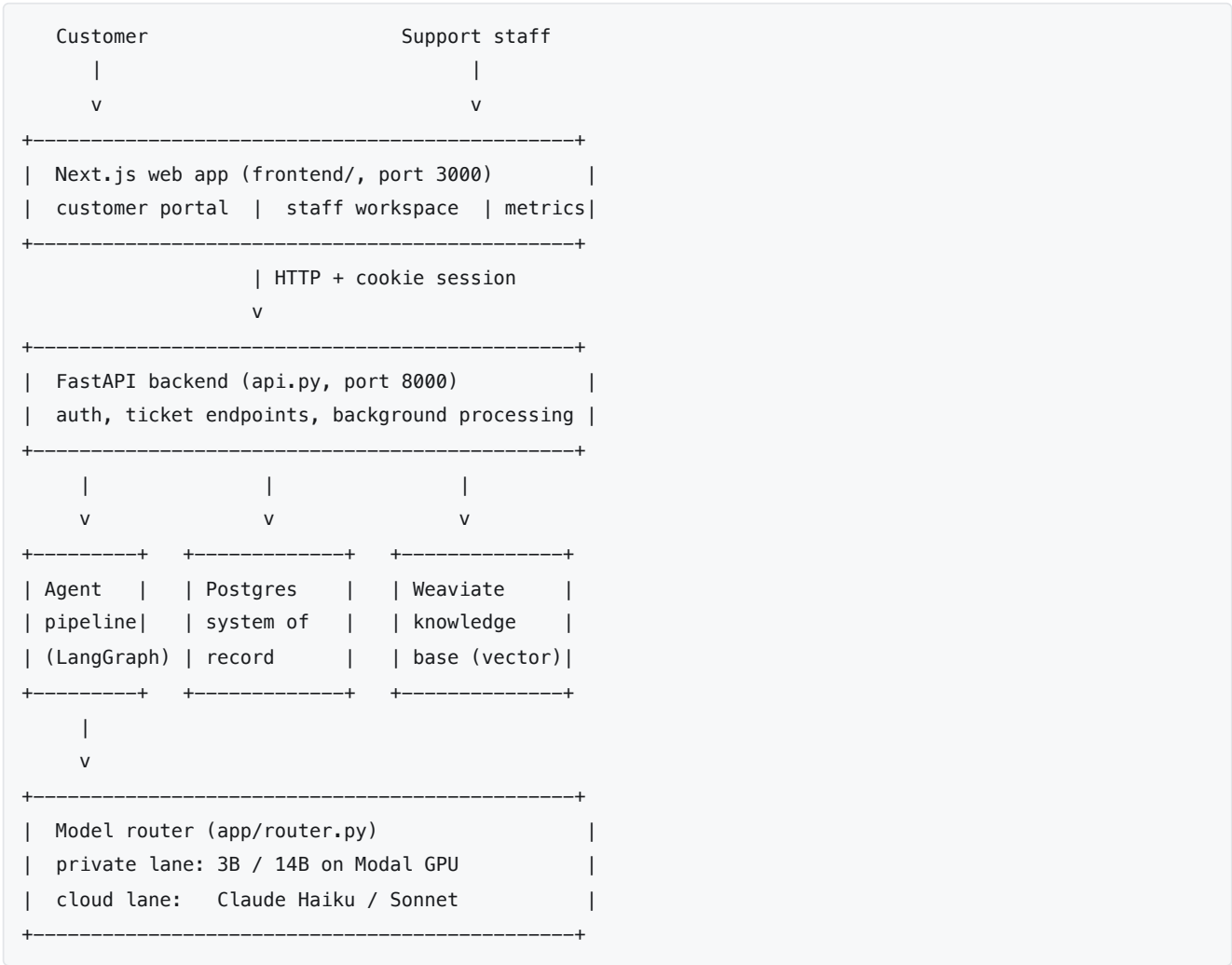
Closed 2026-07-28: the two measurement requirements, classification accuracy and retrieval hit rate, were previously unmeasurable because no labelled evaluation set existed. `data/eval_set.jsonl` (30 labelled tickets) and `eval.py` now report them: category 86.7 percent, priority 73.3 percent, retrieval hit rate 100 percent at five and 36 percent at rank one, with all three critical tickets rated critical after a measured prompt fix. See [10-benchmark-report.md](#) section 4.

Remaining gaps: **cost per ticket** covers GPU time but not cloud tokens; **FR-1's transcript format** has a parser but no endpoint that reaches it; and **FR-8's escalation rule** treats high priority and sensitivity as a higher confidence bar rather than a forced escalation.

02. High-Level Design

Version 1, 2026-07-28.

1. The shape of the system



Inbound channels other than the web form: an email mailbox polled over IMAP, and a Jira project polled for new issues. Both create tickets through the same pipeline.

2. Components

Component	Where	Responsibility
Web app	frontend/	Customer portal (file a request, read the thread), staff workspace (queue, review, approve/reject/edit), metrics dashboard
API	api.py	Authentication, ticket lifecycle endpoints, background pipeline runs, channel dispatch
Deterministic pipeline	app/graph.py	Fixed ten-node LangGraph flow, the demo safety net and the A/B baseline
Autonomous pipeline	app/orchestrator.py , app/agents.py	Five ReAct agents with a twelve-tool universe and a dynamic router node

Component	Where	Responsibility
Model router	<code>app/router.py</code>	Chooses the model from lane and difficulty, caps it by <code>MODEL_TIER</code>
System of record	<code>app/store.py</code> on Postgres	Tickets, threads, tags, links, templates, attachments, metrics
Knowledge base	<code>app/kb.py</code> on Weaviate	Article and past-resolution retrieval by meaning, plus index-back on resolve
Audit chain	<code>app/audit.py</code>	Hash chain over pipeline steps
Channels	<code>app/email_channel.py</code> , <code>app/jira_channel.py</code> , <code>app/adapters.py</code>	Inbound polling and outbound replies

3. The two pipelines

Both are kept, selected by the `AGENT_MODE` environment variable.

- **deterministic** (`app/graph.py`): a fixed sequence of nodes. Fast, predictable, the safety net for a live demo and the control arm for measurement.
- **autonomous** (`app/orchestrator.py` + `app/agents.py`): five ReAct agents that choose tools and steps themselves, with an orchestrator node deciding what runs next. Slower and more variable, but it is the agentic behaviour the brief asks for.

Cross-cutting features are built at the shared layer (`api.py` handlers, `app/store.py`) so both modes inherit them.

Anything touching pipeline internals is changed in both places in the same edit.

4. Request flow, one ticket

1. A customer submits a request. The API writes a pending row immediately and returns a ticket id, so the screen never blocks on a model.
2. A background task runs the pipeline under a 180 second cap, with a second attempt if the first times out (a cold GPU container is the usual cause).
3. The pipeline classifies, checks sensitivity, scores difficulty, picks the lane and model, retrieves knowledge, drafts, reviews against policy, and decides.
4. The decision is either auto-send (the reply leaves through the channel the ticket arrived on) or escalate (a staff member is named, and the customer is told a human is coming).
5. Staff approve, edit, or reject in the workspace. Resolving the ticket archives it and files the resolution back into the knowledge base as precedent.

5. Technology choices

Layer	Choice	Reason
Backend	Python + FastAPI	Mandated stack; async endpoints with background tasks fit the long-running pipeline
Agent wiring	LangGraph	Explicit nodes and conditional edges, so the flow is inspectable rather than implicit
Vector store	Weaviate	Mandated stack; runs locally in Docker with no cloud account
Relational store	Postgres	Durable system of record with JSONB for the whole pipeline state
Frontend	Next.js App Router, TypeScript, Tailwind	Mandated stack

Layer	Choice	Reason
Private model lane	Modal serverless GPU	Sensitive tickets must not reach a third-party API, and no local GPU is available
Cloud model lane	Claude (Haiku and Sonnet)	Strong quality for non-sensitive work at low cost

6. Deployment topology

Everything runs on one developer machine. Postgres and Weaviate run in Docker (`docker-compose.yml`). The two model lanes are remote HTTPS endpoints. There is no clustering, no load balancer, and no separate environments; see [11-nfr.md](#).

Service	Address
Web app	http://localhost:3000
API	http://localhost:8000 (interactive docs at <code>/docs</code>)
Weaviate	http://localhost:8080 , gRPC 50051
Postgres	localhost:5432, database <code>tickets</code>

7. Design principles held throughout

- **The human stays in the loop where it matters.** Auto-send is allowed only for a grounded, policy-passing, high-confidence answer; everything else is a person's decision.
- **Nothing sensitive leaves the building.** The lane decision is made before any model call, and the private lane has no path to a cloud provider.
- **Accumulate, do not overwrite.** Each node adds its own section to the state, so the finished state is the record of how the answer was reached.
- **Fail visibly.** A timed-out or crashed run marks the ticket in an error state rather than leaving it stuck in processing.

03. Low-Level Design

Version 1, 2026-07-28. What is inside each box of [02-hld.md](#).

1. Module map

Module	Lines	Responsibility
api.py	~690	Endpoints, auth wiring, background processing, channel dispatch
app/graph.py	~470	Deterministic pipeline: ten nodes, two conditional edges
app/orchestrator.py	~190	Autonomous pipeline: same nodes as thin wrappers plus a dynamic router
app/agents.py	~295	Five ReAct agents (classify, retrieve, generate, review, learn)
app/tools.py	~130	Tool registry the autonomous agents call
app/router.py	~95	Model choice by lane and difficulty, capped by <code>MODEL_TIER</code>
app/store.py	~710	All Postgres access and the metrics query
app/kb.py, app/embed.py	~46	Vector search, index-back, local embeddings
app/state.py	~65	<code>Ticket</code> model, <code>State</code> shape, confidence policy
app/intake.py	~26	Raw payload to canonical ticket
app/pii.py	~36	Pattern scan for personal data, including a card checksum
app/roster.py	~27	Assignee selection on escalation
app/audit.py	~26	Hash-chain reducer and verifier
app/users.py	~75	Accounts, roles, cookie sessions
app/email_channel.py, app/jira_channel.py, app/adapters.py	~140	Inbound polling, outbound replies, integration seam

2. The state object

One dictionary flows through the pipeline. Each node returns only its own section, and the framework merges them, so the finished object is a record of how the answer was reached.

Key	Written by	Holds
raw_input	caller	The untouched inbound payload
ticket	intake	The canonical ticket (id, source, subject, body, customer, timestamp)
error	intake	Why the payload was rejected, if it was
classification	classify	category, priority, business impact, sentiment
sensitivity	detect_sensitivity	is_sensitive, pii_types, reason
difficulty	score_difficulty	simple or complex, plus a reason
routing	route	lane, tier, intended model
retrieval	retrieve	Ranked knowledge hits with scores

Key	Written by	Holds
draft	generate	reply, kind (answer/question/escalate), confidence
compliance	review	verdict and issues
review_count	review	Caps the regeneration retry
decision	decide	auto_send or escalate, reason, assignee
messages	API layer	The conversation, roles customer / agent / internal
audit	every node	The hash chain, append-only

`audit` uses an append-only reducer, so a node cannot overwrite history even by accident.

3. Deterministic pipeline, node by node

```

START -> intake --> classify -> detect_sensitivity -> score_difficulty -> route
      |           -> retrieve -> generate -> review --> (fail, first time) generate
      |                                           +--> decide -> learn -> END
      +--> (malformed) decide

```

Node	Model call	What it does	Failure behaviour
intake	no	Normalises the payload; rejects an unknown source	Sets <code>error</code> , jumps straight to <code>decide</code>
classify	yes	Four labels from a definition-led prompt, lower-cased	A parse failure raises and the run is retried by the caller
detect_sensitivity	yes	Pattern scan, plus a sensitive-category check, plus a model judgement; any one of them makes it sensitive	If the model output will not parse, the pattern scan and category check still stand
score_difficulty	yes	simple or complex	as classify
route	no	Lane from sensitivity, tier from difficulty, model from the 2x2 grid	Pure code, cannot fail
retrieve	no	Vector search, hits below the relevance floor of 60 are dropped	An empty list is a valid result
generate	yes	Drafts the reply; the first line is a control line carrying kind and confidence	No control line means the whole output is treated as an answer
review	yes	Placeholder check, sign-off check, outbound personal-data scan, then a policy judgement	Skipped when there is no draft
decide	no	The escalate-or-send rule below	Pure code
learn	no	Deliberately does nothing; knowledge write-back happens on resolve	n/a

4. The decision rule

In order, first match wins:

1. Malformed intake, compliance failure, critical priority, or the agent declaring it needs a human, all escalate.
2. A real answer that passed review, whose **grounded confidence** clears the bar, auto-sends.
3. The same answer below the bar escalates with the numbers in the reason.

4. A question back to the customer auto-sends.
5. Anything else escalates.

Grounded confidence is the average of the model's self-reported confidence and the top retrieval score, so both must be high. The bar is 85 for money categories or high priority, 60 otherwise. On escalation an assignee is chosen from the roster by category and priority.

These are heuristics, not calibrated probabilities. Calibrating them needs the labelled set named in [01-requirements.md](#).

5. Autonomous pipeline

The same stations exist, but each is a ReAct agent: it thinks, optionally calls a tool, reads the result, and repeats up to eight steps before it must answer. An orchestrator node decides which station runs next rather than following fixed edges.

The twelve tools are read-mostly lookups (customer record, order, orders by email, billing history, subscription, account status, past tickets, service status, refund eligibility, knowledge search) plus two write actions (request a refund, cancel a subscription) which require a confirmation code derived from the ticket id.

Casing trap worth knowing: the deterministic path lower-cases every classification field, the autonomous path returns the model's raw JSON, so any code reading a classification must lower-case it first.

6. Cross-cutting behaviour in the API layer

- **Background processing with a cap.** The pipeline runs in a thread with a 180 second limit and two attempts. A daemon thread means a hung model call is abandoned rather than blocking shutdown. Both attempts failing sets the ticket to `error`.
- **Cancellation.** Resolving a ticket adds it to a cancelled set, so an in-flight run stops writing rather than resurrecting an archived ticket.
- **Auto-tagging.** After every run, tags are derived from the classification, at the shared layer, so both pipelines get it.
- **Human request override.** A message asking for a person parks the ticket for review with no model call at all.
- **Channel dispatch.** A reply leaves through the channel the ticket arrived on: email over SMTP, Jira as a comment, otherwise in-app only.
- **Session gate.** Every endpoint depends on an active account; customers additionally may only touch tickets born from their own address. Email verification was removed on 2026-07-29 (ADR-014), so the address on an account is asserted rather than proved.

7. Error handling summary

Failure	Behaviour
Malformed payload	Rejected at intake, escalated as "malformed intake"
Model call times out	Second attempt, then ticket marked <code>error</code>
Knowledge base unreachable	Retrieval raises, the run fails and retries; a reviewer signing off is never blocked by it
Email or Jira send fails	Recorded in the response as a delivery string; the reply still stands in the thread

04. Data Model

Version 1, 2026-07-28. Two stores: Postgres holds the system of record, Weaviate holds the searchable knowledge.

1. Postgres, database `tickets`

Tables are created at API startup by `store.init_db()`, which is idempotent: new columns are added with `ADD COLUMN IF NOT EXISTS`, so an existing database migrates itself on boot.

`tickets`

Column	Type	Meaning
<code>ticket_id</code>	TEXT, primary key	<code>T-xxxxxxx</code> while live, <code>HIST-xxxxxxx</code> once archived
<code>subject</code>	TEXT	Ticket title
<code>category</code> , <code>priority</code>	TEXT	Denormalised from the classification, for queue filters
<code>action</code>	TEXT	<code>auto_send</code> or <code>escalate</code> , the pipeline's decision
<code>assignee</code>	TEXT	Staff member named on escalation
<code>human_status</code>	TEXT	<code>pending</code> , <code>approved</code> , <code>rejected</code> , <code>edited</code> , <code>sent</code> , <code>error</code>
<code>lifecycle</code>	TEXT	<code>open</code> , <code>awaiting_customer</code> , <code>resolved</code>
<code>created_at</code> , <code>due_at</code> , <code>resolved_at</code>	TIMESTAMPTZ	Timestamps, stored aware in UTC
<code>tags</code>	JSONB	Auto and manual tags
<code>state</code>	JSONB	The entire pipeline state, including the audit chain and the message thread
<code>csat</code>	INT	Customer rating, 1 to 10, optional
<code>merged_into</code>	TEXT	Set when this ticket was folded into another
<code>customer_email</code>	TEXT	Ownership key for the customer view
<code>processing_seconds</code>	DOUBLE PRECISION	Wall clock of the last pipeline run, feeds the dashboard

The `state` column is deliberately the whole object rather than a normalised expansion: the pipeline shape is still changing, and the audit requirement wants the record kept as produced.

Supporting tables

Table	Columns	Purpose
<code>ticket_links</code>	<code>a</code> , <code>b</code> (composite key)	Related-ticket links, stored once per pair
<code>jira_links</code>	<code>ticket_id</code> , <code>issue_key</code>	Maps a ticket to the Jira issue it came from
<code>templates</code>	<code>id</code> , <code>name</code> , <code>body</code> , <code>category</code> , <code>keywords</code> , <code>auto_use</code>	Canned replies; seeded once if the table is empty
<code>attachments</code>	<code>id</code> , <code>ticket_id</code> , <code>filename</code> , <code>content_type</code> , <code>size</code> , <code>data</code> , <code>created_at</code>	Files stored as bytes, 5 MB ceiling

Accounts

Managed by the auth library in its own table: id, email, hashed password, active flag, verified flag, superuser flag, and a `role` column of `customer`, `staff`, or `admin`. Passwords are stored only as hashes. **The verified flag is inert:** it belongs to the auth library's own model, and since email verification was removed on 2026-07-29 (ADR-014) nothing writes or reads it. It is left in place because dropping a library-owned column would need a migration for no gain.

2. Weaviate, collection `Knowledge`

Property	Meaning
<code>title</code>	Article or past-ticket title
<code>content</code>	The body text that gets retrieved
<code>source</code>	<code>article</code> for curated knowledge, <code>ticket</code> for a past resolution

Vectors are 384 numbers produced locally by a small embedding model, so no text is sent anywhere to be indexed. Search converts cosine distance to a 0 to 100 relevance score and drops anything below 60.

The collection grows by itself: resolving a ticket files the resolution back as a new `ticket` entry, which is the learning loop.

3. Message thread

Threads live inside `tickets.state.messages`, oldest first, each with a role:

Role	Visible to the customer	Written by
<code>customer</code>	yes	The portal, an inbound email, or a Jira comment
<code>agent</code>	yes	An auto-sent reply, or a staff approval
<code>internal</code>	no	Staff notes

One shared filter produces the customer-safe view, so no reader can forget to strip internal notes.

4. Identifiers

Kind	Format
Live ticket	<code>T-</code> plus eight hex characters
Archived ticket	Same id with the prefix swapped to <code>HIST-</code>
Attachment, template	Auto-incrementing integer
Account	UUID

Archiving renames rather than deletes, so a resolved ticket keeps a traceable identity, and reopening renames it back and reconnects its Jira link.

5. Retention and volume

All data is synthetic. Nothing is purged automatically: resolved tickets are archived in place and stay queryable by administrators. Current seeded volume is a few hundred knowledge entries and a few hundred tickets, which is the intended scale, see [11-nfr.md](#).

6. Backup and recovery

Both stores are Docker volumes on the developer machine. There is no scheduled backup; recovery means re-running the seed scripts, which is acceptable because the data is synthetic and reproducible. The commands are in [08-runbook.md](#).

05. API Specification

Version 1, 2026-07-28. Base URL `http://localhost:8000` . A live, always-current version of this is served at `/docs` by the framework; this file is the reviewed narrative version.

1. Authentication

Sessions are a signed cookie named `enklima` , valid for seven days. Every endpoint below except `/` , `/config` and the auth routes requires an **active** account. Email verification was removed on 2026-07-29 (ADR-014), so an account's address is asserted, never proved.

Method	Path	Body	Returns
POST	<code>/auth/register</code>	<code>{email, password}</code>	201 with the new account, always role <code>customer</code> . Opens no session, so the client signs in straight after
POST	<code>/auth/login</code>	form <code>username</code> , <code>password</code>	204 and a session cookie; 400 <code>LOGIN_BAD_CREDENTIALS</code>
POST	<code>/auth/logout</code>	none	204
GET	<code>/users/me</code>	none	200 with the current account

Roles: `customer` (own tickets only), `staff` (the whole live queue), `admin` (staff powers plus templates and the archive).

2. Public

Method	Path	Returns
GET	<code>/</code>	Health, plus which pipeline mode and model tier are active. Instant and dependency-free, for uptime pings
GET	<code>/healthz</code>	The honest health check: probes Postgres and Weaviate and reports each, <code>status</code> is <code>ok</code> only if both answer
GET	<code>/config</code>	Brand name and tagline for the portal
POST	<code>/warm</code>	Wakes the private model lane and returns <code>202</code> immediately without waiting for it. The login screen calls this on load, so the roughly one-minute cold start runs while the visitor is signing in rather than in front of their first ticket. Unauthenticated by necessity, so it is guarded three ways: disabled unless <code>PREWARM=true</code> , rate limited per address, and a global cooldown means that however many people load the page the lane is woken at most once per <code>PREWARM_COOLDOWN_S</code> . Returns <code>{"warmed": false}</code> when it declines

3. Tickets

Method	Path	Who	Notes
POST	<code>/tickets</code>	any signed-in account	Returns immediately with <code>{ticket_id, status: "processing"}</code> ; the pipeline runs in the background. A customer's identity always overrides any address in the payload
GET	<code>/my/tickets</code>	customer	The caller's own tickets
GET	<code>/tickets</code>	staff	The queue. Filters: <code>status</code> , <code>category</code> , <code>tag</code> , <code>q</code> (free text), <code>scope</code> (<code>live</code> for everyone, anything else is admin only, 403 otherwise). Sorting: <code>sort = newest</code> (default), <code>oldest</code> or <code>sla</code> (most urgent deadline first); anything else is 422. Pagination: <code>limit</code> (default and cap 200) and <code>offset</code>

Method	Path	Who	Notes
POST	/tickets/{id}/assign	staff	Manual assignment: {"assignee": "dana"} sets it, {"assignee": null} clears it back to unassigned. Free text on purpose, the pipeline writes short names into the same column. 404 on a missing ticket, 409 if the ticket is resolved
GET	/tickets/{id}	staff	The full stored state
GET	/tickets/{id}/history	staff	That customer's other tickets, unlocked by having this one open
GET	/tickets/{id}/customer	staff	That customer's record: profile, five most recent orders, five most recent charges. Same need-to-know rule as /history, holding this ticket open unlocks this customer and nobody else. These are the lookups the autonomous agents already make as tools, surfaced so a human reviewer is not worse informed than the pipeline that drafted the reply. Each store is read independently, so one being empty or unreachable returns nulls rather than failing the panel. {"profile": null, ...} for a sender who is not in the records, which is normal for email and Jira
GET	/tickets/{id}/thread	owner or staff	The customer-safe message list, internal notes stripped
POST	/tickets/{id}/reply	owner or staff	Adds a customer turn and re-runs the pipeline in the background
POST	/tickets/{id}/resolve	owner or staff	Optional {csat: 1–10}. Archives the ticket, cancels any in-flight run, closes the Jira issue if there is one, and files the resolution back into the knowledge base
POST	/tickets/{id}/reopen	staff	Brings an archived ticket back; 409 if it is not a reopenable archived ticket
POST	/my/tickets/{id}/reopen	customer	Same, for the owner

POST /tickets accepts {subject, body, source, name?, email?}. source must be one of chat, form, email, voice_transcript, jira; anything else is rejected as malformed intake.

4. Review actions

Method	Path	Who	Effect
POST	/tickets/{id}/approve	staff	Sends the draft if there is one, adds it to the thread, and hands the ball to the customer. An escalation with no draft simply records the approval and sends nothing
POST	/tickets/{id}/reject	staff	Marks the draft rejected
POST	/tickets/{id}/edit	staff	{reply} replaces the draft. If the reply was already auto-sent, the thread copy is corrected and the status stays sent
POST	/tickets/{id}/note	staff	{body} adds an internal note, never visible to the customer
POST	/tickets/{id}/tags	staff	{tag}
DELETE	/tickets/{id}/tags/{tag}	staff	Removes it
POST	/tickets/{id}/merge	staff	{duplicate_id} folds that ticket into this one; 400 if the ids are equal, missing, or already merged
POST	/tickets/{id}/link	staff	{other_id} marks two tickets related

A resolved ticket is locked: approve, reject, edit, note and tag all return 409.

5. Templates

Method	Path	Who
GET	/templates , /templates/{id}	staff
POST, PUT, DELETE	/templates , /templates/{id}	admin
POST	/tickets/{id}/apply-template	staff, {template_id} overwrites the draft

6. Attachments

Method	Path	Limits
POST	/tickets/{id}/attachments	staff, multipart file. PNG, JPEG, GIF, WebP, PDF, plain text, CSV only. 5 MB ceiling, 413 above it, 400 on an empty file or a disallowed type
GET	/tickets/{id}/attachments	staff, the list
GET	/attachments/{id}	staff, downloads the bytes

7. Channels and metrics

Method	Path	Who	Notes
POST	/email/sync	staff	Pulls unread mail into tickets; 502 if the mailbox is unreachable. Machine-generated mail is skipped and counted
POST	/jira/sync	staff	Pulls new issues into tickets and remembers the issue key; 502 if Jira is unreachable
GET	/metrics	staff	Volumes, escalation split, compliance pass rate, latency (warm and cold), estimated GPU cost, ratings, and resolution time

8. Status codes used

Code	Meaning here
200, 201, 202, 204	Success, created, accepted, success with no body
400	Bad credentials, invalid merge or link, bad attachment
401	No session, or a session for an account that is no longer active
403	Wrong role, someone else's ticket, archive browsing as non-admin
404	Unknown ticket, template, or attachment
409	The ticket is resolved and locked, or not reopenable
413	Attachment above 5 MB
502	An upstream channel (mail or Jira) failed

06. Decision Log (Architecture Decision Records)

Version 1, 2026-07-28. One record per decision that would be expensive to reverse. Each states the situation, the options weighed, the choice, and what it costs.

ADR-001. Wire the agents with an explicit state graph

Context. Several AI steps must run in order, with branches, and the result must be inspectable afterwards. **Options.** A chain of function calls; a single large prompt doing everything; LangGraph nodes and edges. **Decision.** LangGraph, with each step a named node and each branch an explicit conditional edge. **Consequences.** The flow can be drawn, replayed and audited, and each node is testable alone. The cost is a framework dependency and its state-merging rules to learn.

ADR-002. Keep two pipelines, not one

Context. The brief asks for autonomous agents, but autonomous runs take 60 to 140 seconds per ticket and can wander, which is dangerous in a live demo. **Options.** Ship only the autonomous pipeline; ship only the fixed one; keep both behind a switch. **Decision.** Both, selected by `AGENT_MODE`. The fixed pipeline is the demo safety net and the control arm for measurement. **Consequences.** Every change touching pipeline internals must be made twice, in the same sitting. In exchange there is a real A/B baseline: escalation dropped from 17 percent (fixed) to about 8 percent (autonomous, after tool work), measured on the same twelve tickets.

ADR-003. Decide sensitivity before any model call, and never fall back to cloud

Context. Tickets can carry personal, financial, medical or legal detail. A third-party API must never see those. **Options.** Send everything to the cloud and redact; decide after the first call; decide locally first. **Decision.** A pattern scan plus a category check plus a local model judgement run first, and the routing decision is made from their result. The private lane has no code path to a cloud provider. **Consequences.** A private-lane outage cannot be papered over: the ticket errors rather than leaking. That is the correct failure, though today it stops at an error status instead of escalating to a person, which is a known gap.

ADR-004. Route models on a two-by-two grid rather than by a model deciding

Context. Four model choices exist: small and large open models on the private lane, cheap and strong cloud models. **Options.** Let an agent choose its own model; always use the strongest; a fixed table. **Decision.** Plain code maps (sensitivity, difficulty) to a model, capped by a `MODEL_TIER` switch so day-to-day work runs on the cheapest lane. **Consequences.** Cost and privacy are predictable and provable. The grid is hardcoded rather than configuration, so tuning means a code edit.

ADR-005. Run the private lane on serverless GPU

Context. Sensitive tickets need an open-weight model, and neither laptop has a usable GPU. Corporate policy blocks the vendor's command-line tool on the work machine. **Options.** A cloud provider with a privacy tier; a local model on the laptop; serverless GPU behind a web endpoint. **Decision.** Serverless GPU, deployed from the personal machine, reached by both machines as a plain HTTPS endpoint protected by a shared secret, with a hard spend cap. **Consequences.** The work machine never needs the blocked tool. The costs are cold starts of roughly a minute and real money per wake, which is why measurement runs are batched.

ADR-006. Store the whole pipeline state as one JSON column

Context. The state shape was still moving while the schema had to be stable. **Options.** Normalise every field into columns; store the state as JSON; both. **Decision.** Both, in effect: the fields the queue filters on are real columns, and the entire state, including the audit chain and the thread, is one JSON column. **Consequences.** Schema changes stopped being migrations. The cost is that deep queries go through JSON operators, which is why the metrics query reads a few fields that way.

ADR-007. Prove the audit trail with a hash chain, not a database of logs

Context. The record must show tampering, without new infrastructure. **Options.** Append-only table with permissions; a blockchain; a hash chain in the state. **Decision.** Each entry stores a hash of the previous entry, so editing any past entry breaks every hash after it, and a verifier returns the first broken index. **Consequences.** Tamper evidence for about twenty-five lines of code and no new services. It proves tampering, it does not prevent it.

ADR-008. Learn by indexing resolutions, not by fine-tuning

Context. The system should get better at answering repeat questions. **Options.** Fine-tune a model periodically; index resolved tickets as retrievable precedent. **Decision.** Index-back on resolve, quality-gated, tagged as a ticket source. **Consequences.** Improvement is immediate and reversible, with no training pipeline. Quality now depends on the retrieval floor rather than on model weights.

ADR-009. Write back to the knowledge base on resolve, not on auto-send

Context. An auto-sent reply is a guess until the customer accepts it. **Decision.** Only the resolve action files a resolution back, and only when an agent reply actually exists. **Consequences.** The knowledge base stays clean of unconfirmed answers. The cost is fewer entries.

ADR-010. Hard-gate sign-in on a verified email address

REVERSED 2026-07-29 by ADR-014. Kept here in full because the reasoning still stands and the hole it closed is now open again by choice.

Context. Open signup let anyone claim any address, and a ticket's owner is identified by their address. **Options.** Verify but let unverified users look around; gate reading and filing; gate sign-in itself. **Decision.** Gate sign-in. An account cannot hold a session until the inbox is proved. A failed verification email never blocks signup, and a resend endpoint answers the same way whether or not the address exists, so it cannot be used to discover accounts. **Consequences.** Seeded accounts must be marked verified by the seeder, which was found the hard way: the first version of the gate locked out every seeded staff account.

ADR-011. Three repositories sharing a core package, extraction deferred

Context. The legal and governance systems were copy-forked from this one, so several modules exist three times. **Decision.** Keep three repositories and share through one installable `core` package. The extraction is scheduled after the

current deadline. **Consequences.** Documented in full, with the measured duplication, in `Core-Sharing-Decision.md` in the planning repository.

ADR-014. Remove email verification, reversing ADR-010

Context. ADR-010 gated sign-in on a proved inbox. In practice the gate cost more than it bought for this system: every demonstration needed a real inbox round trip before a customer account could do anything, the mail path is the least reliable dependency in the stack, and the whole feature existed to protect a synthetic support desk with no real customer data in it.

Options. Keep it as built; keep the machinery but stop gating sign-in; auto-verify demo accounts and leave the control real for everyone else; remove it entirely.

Decision. Remove it entirely, by the CEO's explicit call after the alternatives were put to him. Gone: the verification emailer, the `/auth/verify` and `/auth/request-verify-token` routes, `requires_verification` on the login router, the `_require_verified` check on the three customer paths, the verify screen and the check-your-inbox screen. Registering now signs you straight in.

Consequences. The hole ADR-010 closed is open again, and it is a real one: **open signup means anyone can register any address they do not own.** Because a customer's tickets are matched by their account address, a registrant could claim an address belonging to somebody else and see tickets filed from it. This is accepted for a demonstration system with synthetic data and no public deployment; it would have to be closed again before any real use, and re-reading ADR-010 is the way to do that. Two smaller effects: `FRONTEND_URL` became dead configuration and was removed, and the auth library's `is_verified` column is now inert but left in place, since dropping a library-owned column would need a migration for nothing. The suite went from 10 tests to 9: the test asserting an unverified customer cannot file was deleted rather than inverted, because the behaviour it guarded no longer exists.

07. Configuration and Secrets

Version 1, 2026-07-28.

1. Rules

- Every real value lives in a `.env` file at the repository root, which is git-ignored and never committed. The repository carries only `.env.example` with blank placeholders.
- Nothing secret goes in code, in the compose file, or in these documents.
- The lane URL and token are read the moment the model router is imported, so the API will not start without them regardless of which model tier is selected.
- A blank value is not the same as an absent one. Where a default matters, the code falls back on blank as well as missing.

2. Variables

Model lanes

Name	Required	Meaning
<code>PRIVATE_LANE_URL</code>	yes	HTTPS endpoint of the small open-weight model on serverless GPU
<code>REVIEW_LANE_URL</code>	yes	HTTPS endpoint of the larger open-weight model
<code>PRIVATE_LANE_TOKEN</code>	yes	Shared secret both lane endpoints require
<code>MODEL_TIER</code>	no, defaults <code>dev</code>	<code>dev</code> runs everything on the small model; <code>local</code> puts reasoning and hard replies on the larger one; <code>full</code> unlocks the cloud lane
<code>ANTHROPIC_API_KEY</code>	only for <code>full</code>	Cloud lane credential

Application

Name	Required	Meaning
<code>DATABASE_URL</code>	yes	Postgres connection string. Use <code>127.0.0.1</code> rather than <code>localhost</code> to force IPv4
<code>AUTH_SECRET</code>	yes	Signs session cookies and password-reset tokens. Changing it logs everyone out
<code>AGENT_MODE</code>	no, defaults <code>deterministic</code>	<code>deterministic</code> or <code>autonomous</code>
<code>PREWARM</code>	no	<code>true</code> makes the login screen wake the GPU on load, so the cold start runs behind the visitor's typing instead of in front of their first ticket. Off by default: the route is unauthenticated and a wake costs real money
<code>PREWARM_COOLDOWN_S</code>	no	Global floor between wakes, default 240. However many people load the page, the lane is poked at most once in this window
<code>BRAND_NAME</code> , <code>BRAND_TAGLINE</code>	no	The client's branding. Repaints the portal through <code>GET /config</code> , and names the support team every outbound reply is signed with, in both the deterministic and the autonomous pipeline. Defaults to <code>Support</code>
<code>BRAND_SIGNOFF</code>	no	Overrides the sign-off wording outright, for a client whose team is not called "The X Support Team". Derived from <code>BRAND_NAME</code> when unset

Email channel

Name	Required	Meaning
EMAIL_USER , EMAIL_PASSWORD	for mail features	Mailbox credentials. Use an application-specific password, never a primary account password
IMAP_HOST	no, defaults to Gmail	Inbound mail host
SMTP_HOST , SMTP_PORT	no, defaults to Gmail	Outbound mail host

Mail is optional. Without it, ticket intake and replies still work in-app; only the mailbox sync and outbound email are unavailable.

Jira channel

Name	Required	Meaning
JIRA_BASE_URL , JIRA_EMAIL , JIRA_API_TOKEN , JIRA_PROJECT_KEY	for Jira features	Site, account, token, and which project to poll

3. Where secrets actually live

Secret	Home
Lane token	The <code>.env</code> file on each machine, and a platform secret on the GPU service side
GPU platform credentials	The platform's own token file in the user profile, on the personal machine only, never in the repository
Cloud model key	The <code>.env</code> file, only needed for the full tier
Mailbox and Jira tokens	The <code>.env</code> file
Database password	The <code>.env</code> file, and it must match the compose file's value

4. Common configuration mistakes

Symptom	Cause
The API will not start, complaining about a missing key	A lane URL or token is absent
Database authentication failed	DATABASE_URL disagrees with the compose file's credentials. Postgres only applies those on a first initialisation of an empty volume, so the two must be made to agree, and the volume re-initialised if it was created with the old value
Everyone is signed out after a restart	AUTH_SECRET changed
Ticket processing and seeding hang, then fail	A local proxy is intercepting the vector database's gRPC port. Exclude localhost, see 08-runbook.md

5. Spend control

The GPU lane bills per wake, and a cold wake costs real money, so a hard spend cap is set on the platform dashboard and every measurement run is batched into one warm window. Cost estimates per ticket are shown on the metrics dashboard.

08. Runbook

Version 1, 2026-07-28. Operating the system on a developer machine. First-time installation is in the repository `README.md` ; this document is for running, recovering and diagnosing.

1. Daily start

```
docker compose up -d          # Postgres + Weaviate
uv run uvicorn api:app --reload # API on :8000
cd frontend && npm run dev     # web app on :3000
```

One app at a time. All three sibling systems serve their API on :8000 and their front end on :3000, and every front end hardcodes `http://localhost:8000` . Stop this app before starting a sibling, or its front end will silently talk to the wrong API. Database containers have distinct ports and can all stay up.

Behind a proxy that intercepts TLS, set the exclusions **before** the backend and before any seed script, or the vector database calls hang and then fail:

```
export NO_PROXY=127.0.0.1,localhost no_grpc_proxy=127.0.0.1,localhost
```

Stop with `Ctrl-C` in each terminal, then `docker compose down` (without `-v` , which would delete the data).

2. Seeding

Command	Effect
<code>uv run python seed_users.py</code>	Creates the staff, admin and demo customer accounts. Idempotent: existing accounts have their role corrected rather than being duplicated
<code>uv run python seed_kb.py</code>	Creates and fills the knowledge collection
<code>uv run python kb_import.py</code>	Imports the larger corpus of articles and past tickets, offline
<code>uv run python seed_data.py, seed_universe.py</code>	Synthetic customers, orders and tickets for the demo world
<code>uv run python seed_templates.py</code>	Five canned replies, which are what the macro chips on a ticket offer. Idempotent, and skipped names are reported. Without it the macro row renders nothing , so the feature reads as missing rather than unused

After any `docker compose down -v` , all four must be run again.

`uv run python seed_all.py` runs the whole list in the right order, and every step is idempotent, so re-running after a partial failure is safe.

2a. Onboarding a client

Enklima is a product a company installs, so bringing a client on board is a provisioning run rather than a signup screen. One deployment serves one client on purpose: the product's promise is that sensitive tickets never leave the client's own infrastructure, and a shared database would undo that.

```
uv run python provision_client.py --brand Nimbus --tagline "Support that answers"
```

It checks the deployment is configured, refuses with a readable message naming what is missing rather than failing deep inside a seed, runs the same six seeds, and then prints the handover: the brand lines to put in `.env`, the four files holding content the client replaces with its own (`policy.md`, `seed_kb.py`, `app/roster.py`, `seed_data.py`), the channel credentials to add, and the commands to start it.

Add `--skip-seeds` to print the configuration report and the handover without touching any data.

What is product and what is client is the whole reuse story. The pipeline, the agents, the queue, authentication and the SLA logic are the product. The brand, the channels, the knowledge base, the policy, the roster and the thresholds are that client's configuration and content. Nothing about a client lives in the product code, which is why `BRAND_NAME` alone repaints the portal **and** changes the name every reply is signed with.

Rotating the seeded credentials

The seeded passwords are development defaults and **this repository is public**, so anybody can read them. They are fine on a laptop and unacceptable on anything reachable from the internet.

To retire them, supply an override and re-run the same seeder against the deployed database:

```
SEED_ADMIN_PASSWORD=... SEED_STAFF_PASSWORD=... SEED_CUSTOMER_PASSWORD=... DATABASE_URL=<the deployed dat
```

Re-running with an override **rotates** an account that already exists: it replaces the stored hash and keeps the role. Accounts are matched by address, so nothing is duplicated and nothing else is disturbed. Run it with no overrides and it prints a reminder that the published defaults are in use.

3. Health checks

Check	Command or address	Healthy answer
API alive	<code>GET http://localhost:8000/</code>	<code>{"status":"ok", ...}</code> with the active mode and tier
Interactive API docs	http://localhost:8000/docs	The endpoint list renders
Database	<code>docker compose ps</code>	Both containers up
Sign-in works	http://localhost:3000/login	A seeded staff account reaches the workspace
Model lane warm	Submit one ticket and watch the log	<code>[pipeline] T-xxxx done on attempt 1</code>

4. Routine operations

Task	How
Warm the GPU lane before a demo	Submit one throwaway ticket about ten minutes ahead; the first call after idle takes about a minute
Switch pipeline mode	Set <code>AGENT_MODE</code> and restart the API
Switch model tier	Set <code>MODEL_TIER</code> and restart the API
Pull in new mail	<code>POST /email/sync</code> as staff, or the button in the workspace
Pull in new Jira issues	<code>POST /jira/sync</code> as staff
Read the numbers	The metrics page in the workspace, or <code>GET /metrics</code>

5. Incidents

A ticket is stuck at "processing"

The pipeline runs in the background with a 180 second cap and two attempts. If both fail the ticket is set to `error`, which is visible in the queue. Check the API log for `[pipeline] ... attempt N failed`. The usual cause is a cold GPU container; retry once the lane is warm by adding a customer reply, which reprocesses the ticket.

Everything returns 401 after a restart

`AUTH_SECRET` changed, so existing cookies no longer verify. Sign in again.

Database authentication failed on startup

`DATABASE_URL` and the compose credentials disagree. Make them match. If the volume was initialised with the old password, remove only the Postgres volume and let it re-initialise, so the knowledge base volume survives:

```
docker compose down
docker volume rm <project>_pgdata
docker compose up -d
```

Then re-run the seeds.

Retrieval returns nothing

The knowledge collection is empty, usually after a volume wipe. Run `seed_kb.py` then `kb_import.py`.

Seeding or processing hangs on the vector database

A local TLS-intercepting proxy is routing the gRPC port. Set the exclusions in section 1 in the same shell.

Mail or Jira sync returns 502

The upstream is unreachable or the credentials are wrong. Neither blocks the rest of the system; the endpoint reports the failure rather than half-importing.

6. Before a demo, in order

1. `git pull`, then `uv sync` and `npm install` if either lock file moved.
2. Start the databases, the API and the web app.
3. Run the seeds if this machine is fresh.
4. Warm the model lane with one throwaway ticket.
5. Sign in as staff and confirm the queue renders and the metrics page has numbers.
6. Keep `AGENT_MODE=deterministic` unless the autonomous behaviour is what is being shown: it is several times faster and does not wander.

7. What is deliberately not here

No monitoring, alerting, log aggregation, scheduled backup, or on-call procedure. This is a single-machine demonstration system, see [11-nfr.md](#).

09. Test Plan and Report

Version 1, 2026-07-28.

1. Strategy

Three layers, in increasing cost:

Layer	Cost	What it proves
Automated tests (<code>tests/</code>)	free, no model calls, no database	The gates: who may do what, what a resolved ticket refuses, what approve actually sends
Live probes against a running instance	free	The auth chain end to end, through real HTTP
Benchmark runs (<code>bench.py</code>)	GPU money	Behaviour of the whole pipeline on a fixed ticket set, both modes. See 10-benchmark-report.md
Labelled evaluation (<code>eval.py</code>)	free, or one model call per ticket	Whether the decisions were right : classification accuracy and retrieval hit rate against 30 labelled tickets

The automated layer runs with the application driven directly, with the authentication dependency and the store replaced, so no Postgres, no vector database and no model are needed. That is what keeps it free and fast.

2. Running them

```
uv run --with pytest --with httpx python -m pytest tests/ -q
```

`pytest` and `httpx` are not project dependencies, hence `--with` .

3. What is covered today

Test	Pins
<code>test_approve_escalation_sends_nothing</code>	Approving an escalation with no draft sends nothing, the historical bug it was written for
<code>test_approve_draft_sends_and_hands_ball_to_customer</code>	Approving a real draft adds it to the thread, dispatches it, and moves the ticket to awaiting-customer
<code>test_approve_resolved_is_locked</code>	A resolved ticket refuses further action with 409
<code>test_customer_can_file</code>	A customer can create a ticket, and their identity comes from the account rather than the form
<code>test_archive_scope_is_admin_only</code>	Archive browsing is 403 for staff
<code>test_staff_reopen_clears_the_doomed_mark</code>	Reopening cancels the resolve-time cancellation, so the ticket can process again
<code>test_reopen_refuses_non_archived</code>	Reopening a live ticket is 409
<code>test_customer_cannot_reopen_someone_elses_ticket</code>	Ownership is enforced on reopen
<code>test_customer_history_needs_a_real_ticket</code>	The history panel refuses an unknown ticket
<code>test_window_blocks_then_recovers</code>	The rate limiter refuses the call over the cap with 429, then allows again once the window has passed

Test	Pins
<code>test_keys_are_independent</code>	One account's spent rate budget never affects another's
<code>test_assign_and_clear</code>	Manual assignment sets a trimmed assignee and null clears it back to unassigned
<code>test_assign_missing_ticket_404s</code>	Assigning on an unknown ticket refuses with 404
<code>test_unknown_sort_rejected</code>	A sort key outside the whitelist is refused with 422 and never reaches SQL
<code>test_sla_sort_reaches_store_as_the_whitelisted_key</code>	The <code>sla</code> sort arrives at the store as the whitelisted key, not as user text

Last run: 2026-08-02, 15 passed, 0 failed, 0.7 seconds.

4. The auth probe, and why it no longer applies

A seventeen-check live probe ran on 2026-07-28 against the running API, with mail sending swapped for a recorder. It covered the email verification chain end to end: signup starting unverified, an unverified sign-in refused with its own distinct reason, the resend endpoint answering identically for real and unknown addresses so it could not be used to discover accounts, the link built correctly, the token accepted once and refused on replay or tampering, and a verified customer reaching their own requests but not the staff queue. It caught a real defect at the time: the gate initially locked out every seeded staff account.

All of that was removed on 2026-07-29 (ADR-014), so the probe no longer describes this system. It is recorded here rather than deleted because it is the evidence for what the removal gave up. What still holds from the same session: a wrong password returns the generic bad-credentials reason, open signup cannot mint a staff role, and seeded staff sign in and reach the queue.

5. Known gaps

These are named rather than hidden, and are the honest answer if a reviewer asks what is untested.

Gap	Why it matters
No unit tests for the model router or the decision rule	Those two functions encode all the judgement in the system
No golden-file tests for intake parsing	A parser change could silently alter the canonical ticket
No mocked-model contract tests per agent	Agent output shape changes would only surface in a live run
No end-to-end test with a fake model	The full graph is only exercised by paid runs
No tamper test on the audit chain	The verifier exists but nothing proves it fires
No test asserting sensitive tickets never reach the cloud lane	This is the system's strongest privacy claim and it rests on reading the router, not on a test
No labelled evaluation set	Closed 2026-07-28: <code>data/eval_set.jsonl</code> holds 30 labelled tickets and <code>eval.py</code> scores them. The retrieval half is free to re-run at any time; the classification half costs one model call per ticket. Results in 10-benchmark-report.md section 4

The first six are a few hours of free work each. The last needs data creation and is the largest single item.

6. Test data

Everything is synthetic: seeded customers, orders, knowledge articles, and a twelve-ticket benchmark batch. No real customer data has ever been in the system.

10. Benchmark Report

Version 1, 2026-07-28. Numbers as measured, with their limits stated. Raw results are committed as `bench_baseline.json`, `bench_deterministic.json` and `bench_autonomous.json`.

1. Method

`bench.py` runs the same twelve synthetic tickets through the whole pipeline and records, per ticket: the classification, the decision and its reason, the compliance verdict, the model's self-reported confidence, the top retrieval score, the blended grounded confidence, wall-clock latency, and for the autonomous mode the tools used and the number of steps.

All three runs used the larger open-weight model on the private lane (`MODEL_TIER=local`), so the cloud lane is not represented in these figures.

2. Results

Run	Mode	Escalated	Auto-sent	Compliance passed	Mean confidence	Mean latency
<code>bench_deterministic.json</code>	fixed pipeline	16.7% (2 of 12)	83.3%	12 of 12	88	18s (16 to 23)
<code>bench_baseline.json</code>	autonomous, before tool work	33.3% (4 of 12)	58.3%	11 of 12	89	91s (45 to 113)
<code>bench_autonomous.json</code>	autonomous, after tool work	0%	100%	12 of 12	89	90s (41 to 132)

Every escalation in both of the first two runs was for the same reason: grounded confidence just under the bar (87, 84, 89, 85 against a threshold of 90). None were compliance failures.

3. What these numbers mean

- **The autonomous pipeline started worse than the fixed one and ended better.** Giving the agents a real tool universe took escalations from a third of the batch to none, against a fixed-pipeline control of one in six. That comparison is the point of keeping both pipelines.
- **Escalation was driven by the confidence bar, not by bad answers.** Answers scoring 87 out of 100 were being sent to a human. The bar for sensitive and money categories has since been lowered from 90 to 85 for exactly this reason, so a rerun should show fewer escalations in both modes.
- **Autonomy costs about five times the wall clock.** Roughly 90 seconds against 18. That is the trade the mode switch exists to let you make, and it is why the fixed pipeline is the demo default.

4. Labelled evaluation, 2026-07-28

The benchmark above measures what the system **decided**. It cannot say whether the decision was **right**, because those twelve tickets have no correct answer attached. A separate labelled set now closes that gap.

`data/eval_set.jsonl` holds 30 tickets with the correct category, priority, difficulty and sensitivity written down, and, for the 22 that a knowledge-base article genuinely answers, the titles of the acceptable articles. The remaining 8 are deliberately not covered by the knowledge base: two complaints, two feature requests, an outage, a legal threat, an erasure request, and one too vague to answer.

`eval.py` scores it in two halves, separable by cost:

```
uv run python eval.py          # retrieval only, free, no model calls
uv run python eval.py --classify # adds one model call per ticket
```

Results, 30 tickets, 0 errors:

Measure	First run	After the priority-prompt fix
Retrieval hit rate, top 5	100% (22 of 22 answerable)	100%
Retrieval hit at rank 1	36% (8 of 22), mean rank 1.86	36%
Classification accuracy, category	83.3%	86.7%
Classification accuracy, priority	66.7%	73.3%
Both correct on the same ticket	60.0%	66.7%
Critical tickets rated critical	1 of 3	3 of 3

What the fix was. The first run showed the real defect: an office-wide outage and a legal threat both came back as merely high. The priority definitions were doing too little work, so they were rewritten in both pipelines to say what critical actually means (many users affected, the account compromised, lawyers or regulators named, safety at risk), to say that one stuck customer is not many users, and to stop "money is at stake" pulling every ordinary billing question up to high. Two rounds of that, measured each time, moved priority from 66.7 to 73.3 percent and fixed all three critical tickets. Nothing but prompt text changed.

Reading these honestly:

- **Hit rate at five is a soft measure.** With five slots and a relevance floor, the right article is almost always somewhere in the list. Rank one at 36 percent is the number that discriminates, and it is the one to improve.
- **Retrieval cannot tell "no good article" from "good article".** All eight no-coverage tickets still returned hits, scoring 77 to 89, inside the same band as genuine matches. The score floor cannot separate them. What actually protects those tickets is the control line in the drafting prompt, which lets the agent say it must ask or escalate. That is prompt-level, not structural.
- **Category confusion is sensible, not random.** Four errors, each a single ticket: an account problem read as technical, a billing question read as general, a complaint read as a refund, and a feature request read as billing.
- **Priority still errs upward.** Eight misses: six over-rate (five medium to high, one low to medium) and two under-rate (medium read as low). Over-rating costs a needless human review; under-rating is the direction that hurts, and the worst cases, the criticals, are now correct.
- **Small deltas here are noise.** Thirty tickets against a model at temperature zero is still one sample, and one run lost a ticket to a lane timeout. Treat a two-point move as nothing; the critical-ticket count moving from 1 of 3 to 3 of 3 is the real result.

5. Limits, stated plainly

- **Twelve tickets, and thirty labelled ones, are batches rather than samples.** No confidence intervals are claimed and none should be.
- **The labels were written by the same person who built the system,** which is the strongest bias in the evaluation numbers.
- **Latency depends on a warm GPU container.** A cold start adds roughly a minute and is excluded from these figures by warming the lane first.
- **The runs predate the confidence recalibration.** The reported escalation rates are against the old bar of 90.
- **Compliance pass rate is close to a ceiling** at 11 or 12 of 12, so it discriminates poorly; it is reported because it is a stated requirement metric.

6. Live system metrics

Separately from the benchmark, the running system computes its own numbers over whatever is in the database, shown on the metrics dashboard and available at `GET /metrics`: ticket volume, escalated against auto-resolved, compliance reviewed and passed, average processing time split into warm and cold, an estimated GPU cost per ticket, average customer rating, and average time to resolution. Those are live operational figures, not a benchmark, and they move as the demo data changes.

7. Reproducing

```
uv run python bench.py          # decision behaviour, both modes, the committed ticket set
uv run python eval.py           # retrieval hit rate, free
uv run python eval.py --classify # + classification accuracy, one call per ticket
uv run python eval.py --classify --limit 5 # cost fence
```

Both benchmarks cost GPU money on every wake, except `eval.py` without `--classify`, which is free. Warm the lane first and run a whole batch in one window rather than one ticket at a time. Raw results are written to `bench_*.json` and `eval_results.json`.

Lane latency: why there is no "use the small model for cheap steps" option

Measured 2026-08-03 against the live private lane, both containers warm, median of three calls on a realistic tool-selection prompt:

output tokens	3B on T4	14B on A10G	ratio
128	7.4s	9.6s	the 14B is 1.30x slower
512	14.8s	12.7s	the 14B is faster

Every agent loop runs at `max_new_tokens=512`, so at the size that matters **the larger model is the faster one**. The bottleneck is the hardware, not the parameter count: the 3B sits on a T4 and the 14B on an A10G.

A change to route "cheap" tool-selection steps to the 3B was written, measured, and **removed before release** on this evidence. It would have cost latency and answer quality at the same time. Recorded because the intuition it contradicts, smaller means faster, is the obvious one and would otherwise be tried again.

An 8-token probe run first showed the 14B at 1.3s against the 3B at 2.6s, which overstates the effect: at that length the call is all overhead and no decoding. The 512-token figure is the honest one.

11. Non-Functional Requirements

Version 1, 2026-07-28. Targets with numbers, and what is actually measured against them.

1. Performance

Property	Target	Measured	Source
Ticket handling, fixed pipeline	seconds, not minutes	mean 18s, range 16 to 23	benchmark, warm lane
Ticket handling, autonomous pipeline	allowed to be slower	mean 90s, range 41 to 132	benchmark, warm lane
First call after the GPU lane goes idle	under two minutes	roughly 60 to 90 seconds	live probes
Hard ceiling per run	180 seconds, two attempts, then a visible error	enforced in code	PIPELINE_TIMEOUT_S
Interface responsiveness	the submit screen never waits on a model	met by design: the ticket is saved and the pipeline runs in the background	POST /tickets

Latency is recorded per ticket and shown on the dashboard split into warm and cold runs, so the GPU cold-start cost stays visible rather than averaged away.

2. Scale

Property	Target
Concurrent users	one, a single developer machine
Ticket volume	low hundreds, synthetic
Knowledge corpus	a few hundred entries
Concurrency inside the pipeline	one ticket at a time per run; parallel model calls are deliberately serialised on the single GPU container

This is a demonstration system. High availability, horizontal scaling, and queueing under load are explicitly out of scope.

3. Availability

There is no availability target. Everything runs on one machine, both data stores are Docker volumes, and there is no failover. The relevant guarantee is narrower and is met: **no single failure loses a ticket or leaves it stuck**. A failed run marks the ticket in error, a cancelled run cannot resurrect an archived ticket, and archiving happens before the knowledge write-back so a retrieval failure cannot strand a resolution.

4. Privacy and security targets

Requirement	Status
A sensitive ticket never reaches a third-party API	Met by construction: the lane decision precedes every model call and the private lane has no code path to a cloud provider
No fallback from the private lane to the cloud when it is down	Met. The failure surfaces as an error rather than a silent downgrade

Requirement	Status
Outbound replies carry no personal data	Enforced by a scan of the draft during review
Only active accounts hold a session	Met. Email verification was removed 2026-07-29 (ADR-014), so an address is asserted, not proved
Customers see only their own tickets, staff never see raw archives	Met

Full detail in [12-security-review.md](#) and [13-privacy-and-data-handling.md](#).

5. Auditability

Every pipeline step appends to a hash chain stored with the ticket, and the whole state is persisted as produced. Editing any past entry breaks every hash after it and is detectable. The chain proves tampering; it does not prevent it, and there is no external notarisation.

6. Maintainability and reuse

The module boundaries were designed so the skeleton could be reused, and that has been proved twice: both the legal-review and governance systems were built from this shape. The cost of that reuse by copying is measured, and the plan to replace it with a shared package is recorded in the planning repository's core-sharing decision.

7. Cost

The GPU lane bills per wake rather than per token, so the practical unit is a warm window, not a request. A hard spend cap is set on the platform, measurement runs are batched into one window, and the dashboard shows an estimated GPU cost per ticket. Cloud token cost is not tracked, so full-tier runs under-report total cost. That gap is known and named.

8. Compliance and data residency

Not applicable at this stage: all data is synthetic, nothing is regulated, and there is no production deployment. The privacy lane exists so that the design does not have to change when real data appears.

12. Security Review

Version 1, 2026-07-28. A threat model for a single-machine demonstration system: what an attacker would try, and what stops them today.

1. Assets worth protecting

Customer ticket content and threads; account credentials; the model lane token; the mailbox and Jira credentials; the audit chain's integrity.

2. Trust boundaries

Boundary	Who is on the other side
Browser to API	Anyone who can reach port 8000
API to model lanes	An internet-reachable GPU endpoint, and optionally a cloud model provider
API to mailbox and Jira	Third-party services holding real credentials
API to data stores	Local Docker containers

3. Threats and controls

#	Threat	Control today	Residual risk
1	Someone signs up claiming an address they do not own, then reads that person's tickets	Only partly mitigated. Ticket ownership is matched on the account address, never on a form field, so a form cannot be used to claim someone else's tickets. But since email verification was removed on 2026-07-29 (ADR-014), registering the address itself is unchecked	This is the largest open hole in the system. It was closed on 2026-07-28 and deliberately reopened; see R-11 in the risk register. Restoring the gate is the fix, and ADR-010 records how it was built
2	Account enumeration through the resend endpoint	It answers 202 for every address, real or not	None material
3	Password guessing	Passwords are stored hashed by the auth library	No rate limiting or lockout. This is the clearest missing control
4	Session theft	Signed cookie, seven-day life, secret in the environment	The cookie is not marked secure because the demo runs over plain HTTP locally. That must change before any real deployment
5	A customer reads someone else's ticket	Every ticket route checks ownership against the account address	None known
6	Staff browse the whole archive	Archive scope is administrator-only, and staff instead get need-to-know access to one customer's history from a live ticket	An administrator can read everything, by design
7	Sensitive ticket content leaks to a third-party model	The lane decision is made before any model call and the private lane cannot reach a cloud provider	Detection of sensitivity is heuristic: a pattern scan, a category rule and a model judgement. A novel phrasing could be misjudged
8	An outbound reply leaks personal data	The draft is scanned during review and a hit fails the review	Same heuristic limit

#	Threat	Control today	Residual risk
9	The lane token is stolen and someone else spends the GPU budget	Shared-secret token, hard spend cap on the platform	The endpoint is internet reachable; the cap is the real backstop
10	The audit trail is edited to hide an action	Hash chain, with a verifier that returns the first broken entry	Tamper evident, not tamper proof. Nothing external notarises the chain
11	Malicious file upload	Type allowlist, 5 MB ceiling, files stored as bytes and served with the stored content type	Files are not virus scanned, and are served back with their declared type
12	Prompt injection through ticket text, aiming to make the agent misbehave or call a tool	Write tools require a confirmation code derived from the ticket; the compliance review inspects the draft before anything is sent; the human gate stands for everything not auto-sent	Not systematically tested. A crafted ticket steering retrieval or tone is plausible
13	Denial of service by flooding tickets	A per-run time cap stops one ticket jamming the queue	No rate limiting on ticket creation
14	Secrets committed to the repository	Secrets live only in a git-ignored environment file; the repository holds blank placeholders	A careless paste remains possible; nothing scans commits
15	Company metrics read without signing in	The pre-auth <code>/metrics</code> page, which fetched and rendered company metrics to any visitor with no cookie, was deleted on 2026-07-28 along with a duplicate pre-auth ticket page. Metrics now live only behind the staff workspace	None known. The API's own metrics route still requires a session

4. What would have to change before real users

In priority order:

1. Rate limiting and lockout on sign-in and on ticket creation.
2. HTTPS everywhere, with the session cookie marked secure.
3. A tested policy for prompt injection, including tool-call review.
4. Virus scanning of attachments, and serving them with a forced download type.
5. Secret scanning in the commit path, and rotation of the seeded development credentials.

5. Deliberate non-goals

No penetration test has been performed. No dependency vulnerability scanning is in place. There is no authorisation model beyond the three roles. All of this is consistent with a demonstration system on synthetic data, and none of it should be read as production readiness.

13. Privacy and Data Handling

Version 1, 2026-07-28.

1. Position

All data in this system is synthetic. No real customer information has ever been processed. The privacy design exists so that the architecture does not have to change if real data ever arrives, and so the privacy claim can be demonstrated rather than asserted.

2. What personal data the system touches

Data	Where it comes from	Where it is stored
Name and email address	The account, or an inbound email or Jira issue	The accounts table and the ticket row
Ticket subject and body, which may contain anything the customer typed	The portal, a mailbox, or Jira	The ticket row, inside the stored state
Conversation thread	Replies from both sides, plus internal staff notes	Inside the stored state
Attachments	Staff upload	The attachments table, as bytes
Satisfaction rating	The customer on resolve	The ticket row

Internal notes are stored in the same thread but carry a role that the customer-safe view filters out, through one shared function so no reader can forget.

3. The sensitivity decision

Before any model is called, three checks run:

1. A pattern scan for email addresses, phone numbers, national identifier formats, and card numbers validated by checksum.
2. A category rule: refunds, billing and account tickets are treated as sensitive by default.
3. A model judgement on the ticket text, covering financial, identity, health, authentication, legal and protected-trait information.

Any one of the three makes the ticket sensitive. The result is recorded on the ticket with its reasons.

4. Where data flows

Flow	Sensitive ticket	Non-sensitive ticket
Classification and reasoning	Self-hosted open-weight model on a private GPU endpoint	Same, by default
Reply drafting	Private GPU endpoint only	Private endpoint, or a cloud model when the full tier is enabled
Retrieval and embedding	Local vector database, embeddings computed on the machine	Same
Storage	Local Postgres	Same

Nothing sensitive is ever sent to a third-party model provider. That is enforced by the routing table: the private lane maps only to the self-hosted endpoints, and there is no fallback path from private to cloud. If the private lane is unavailable, the ticket fails visibly rather than being downgraded.

The one qualification worth stating: the private lane is self-hosted on rented GPU infrastructure, not on premises. The model weights and the runtime are ours and no provider trains on the traffic, but the data does leave the local machine over an authenticated HTTPS connection to that endpoint.

5. Outbound leak control

Before any reply is judged compliant, the draft is scanned with the same pattern set. A reply that carries personal data fails review, which either forces a regeneration or escalates it to a human. This is the control against the model echoing sensitive input, or pulling another customer's detail out of retrieved material.

6. Retention

Data	Retention
Live tickets	Kept indefinitely
Resolved tickets	Archived in place under a new identifier, still queryable by administrators
Resolutions filed into the knowledge base	Kept indefinitely, and retrievable for future tickets
Accounts	Kept until deleted by hand
Attachments	Kept with the ticket

There is no automatic deletion, no retention schedule, and no subject-access or erasure workflow. On synthetic data that is acceptable; with real data it would not be, and it is the first thing to build.

7. Access

Three roles. A customer reaches only tickets matching their account address, which since 2026-07-29 is asserted at signup rather than proved (ADR-014). Staff reach the live queue and, from a live ticket, that one customer's past tickets. Administrators reach everything including the raw archive and the templates. Internal notes are never exposed to customers by any route.

8. Learning loop and privacy

Resolving a ticket files the resolution into the knowledge base, where it can be retrieved for later tickets from other customers. Today the entry is the reply text that was actually sent, which had already passed the outbound personal-data scan. That scan is therefore doing double duty: it protects the customer receiving the reply, and it keeps the shared knowledge base clean. If real data were introduced, this loop would need an explicit redaction step rather than relying on that scan.

9. Gaps to close before real data

1. A retention schedule and a deletion path, including from the knowledge base.
2. An explicit redaction step on the learning write-back.
3. Encryption at rest for both stores, and HTTPS in transit for the browser session.
4. A record of which model saw which ticket, retained for audit.

14. Model Card

Version 1, 2026-07-28. What models this system uses, how they are chosen, what they are good at, and where they fail.

1. Models in use

Lane	Model	Host	Used for
Private, simple	Qwen2.5-3B-Instruct	Serverless GPU (T4), half precision	Everyday reasoning and replies on the cheapest tier
Private, complex	Qwen2.5-14B-Instruct	Serverless GPU (A10G), 4-bit quantised	Reasoning checks and harder replies, and all sensitive work that needs quality
Cloud, simple	Claude Haiku 4.5	Provider API	Non-sensitive simple replies, full tier only
Cloud, complex	Claude Sonnet 4.6	Provider API	Non-sensitive complex replies, full tier only
Embeddings	A small local sentence embedding model, 384 dimensions	The machine itself	Turning text into vectors for retrieval. No text leaves the machine to be indexed

All generation runs at temperature zero, so repeated runs are as reproducible as the model allows.

2. How a model is chosen

Two questions, asked in plain code, never by a model:

1. Is the ticket sensitive? That picks the lane, and it is decided before any model call.
2. Is it simple or complex? That picks the size within the lane.

A `MODEL_TIER` switch then caps the result: `dev` forces everything onto the small model, `local` allows the larger open model, `full` unlocks the cloud lane. The interface always shows the model the grid intended, even when the tier caps what actually ran, so the trade-off is visible rather than hidden.

3. What the models are asked to do

Task	Output contract
Classify	Strict JSON: category, priority, business impact, sentiment
Score difficulty	Strict JSON: simple or complex, plus a reason
Judge sensitivity	Strict JSON: a boolean, matched categories, a reason
Draft a reply	A control line carrying kind and confidence, then the reply text
Review the draft	The single word PASS, or FAIL with a reason
Learn from a resolution	Whether the resolution is worth filing, and its title

Every one of these is parsed by code that expects a specific shape, and every parser tolerates the model wrapping its answer in prose or fences.

4. Known failure modes

Failure	How it shows	What contains it
Invalid JSON from a small model	A parse error, the run retries, and a second failure marks the ticket in error	The retry, plus the pattern and category checks that stand on their own for sensitivity
Truncated output on long answers	The reply stops mid-sentence	Generous token limits per call; still possible on unusually long threads
Over-escalation	Correct answers sent to a human because confidence sat just under the bar	The bar was recalibrated from 90 to 85 after answers scoring 87 were being escalated
Third-person voice, or internal wording copied into a customer reply	It reads like notes rather than a letter	An explicit voice rule in the prompt and a quality clause in the review gate
Hallucinated detail when the knowledge base has nothing	An invented answer	The prompt requires the model to ask a question or decline instead, and the retrieval floor drops weak matches. This is prompt-level, not structural, so it depends on model obedience
Wandering in autonomous mode	Many steps, repeated tool calls, long latency	An eight-step ceiling per agent, a duplicate-call block, and a whole-run time cap
Cold container	The first call after idle takes about a minute	The lane is warmed before demos, and the caller retries once

5. What is not measured

Measured as of 2026-07-28, against 30 labelled tickets: category 86.7 percent, priority 73.3 percent, both correct on the same ticket 66.7 percent, retrieval hit rate 100 percent at top five and 36 percent at rank one. All three critical tickets are rated critical; the remaining priority errors lean upward, six over-rating against two under-rating, which costs a needless human review rather than a missed emergency.

The first measured run scored only one of the three criticals correctly, which is what prompted rewriting the priority definitions in both pipelines. That change is the clearest evidence in the project that the labelled set earns its keep: the defect was invisible until something scored it.

Still unmeasured: whether an auto-sent reply was actually **useful** to the customer, and whether an escalation was necessary. Both need either a human rater or real customer feedback, and neither exists on synthetic data. The customer rating field is the closest available proxy and is only populated in demonstrations.

6. Appropriate and inappropriate use

Appropriate: drafting and triaging synthetic customer requests with a human able to review anything the system did not auto-send.

Inappropriate without further work: any decision with legal or financial consequence taken without review; use on real customer data before the retention and redaction gaps in [13-privacy-and-data-handling.md](#) are closed; any setting where a wrong auto-sent reply cannot be corrected.

7. Human oversight

Auto-send is deliberately narrow: a policy-passing answer, grounded in retrieved material, above a confidence bar that is higher for money and high-priority tickets. Everything else reaches a person, who can approve, edit or reject, and an explicit request to speak to a human bypasses the model entirely.

15. Risk Register

Version 1, 2026-07-28. Live risks only. A risk that has been closed is recorded as closed rather than deleted, because the mitigation is often the interesting part.

Likelihood and impact are low, medium or high. Exposure is the pair read together.

Open risks

#	Risk	Likelihood	Impact	Mitigation	Owner
R-1	The GPU lane is cold or unavailable during a live demonstration, so tickets take minutes or fail	Medium	High	Warm the lane about ten minutes before, run the fixed pipeline rather than the autonomous one, and keep a recorded walkthrough as a fallback	Build owner
R-2	GPU spend exhausts the budget mid-work	Medium	Medium	A hard cap on the platform, batched measurement runs, and a per-ticket cost estimate on the dashboard	Build owner
R-3	Priority is judged correctly about three quarters of the time	Medium	Low	Measured on the labelled set, and improved from 66.7 to 73.3 percent by rewriting the priority definitions. All three critical tickets now rate critical, and the residual errors over-rate rather than under-rate. Escalation never depends on priority alone	Build owner
R-4	Sensitivity detection misjudges a novel case, and a sensitive ticket is drafted on the cloud lane in full tier	Low	High	Three independent checks, any one of which is enough; the default tier keeps everything on the private lane anyway. A dedicated test asserting this is still missing	Build owner
R-5	Prompt injection through ticket text steers a reply or a tool call	Medium	Medium	Write tools need a confirmation code, the compliance gate inspects every draft, and anything not auto-sent reaches a human. Not systematically tested	Build owner
R-6	A model output shape changes and a parser silently degrades	Medium	Medium	Every parser is defensive and failures are visible as errors rather than silent. Mocked contract tests per agent do not yet exist	Build owner
R-7	Duplicated code across the three systems means a bug fixed here is left unfixed there	Medium	Medium	The duplication is measured and the shared-package plan is written; two such bugs have already occurred and were fixed in both places	Build owner
R-8	Corporate security tooling on the work machine breaks local development in new ways	Medium	Low	Known workarounds are documented: exclude localhost from the proxy, and never write credential-shaped strings through the editor	Build owner
R-9	Documentation drifts from the code as the last week of changes land	Medium	Low	Every document in this set is dated and states what it was checked against; the traceability matrix is the piece to re-check first	Build owner
R-10	No rate limiting on sign-in or ticket creation	Low here, High in production	Medium	Accepted for a single-machine demonstration, named as a blocker for real use in the security review	Build owner
R-11	Anyone can sign up claiming an address they do not own, and read the tickets filed from it	Low here, High in production	High	Currently unmitigated by choice. This was closed on 2026-07-28 by gating sign-in on a verified inbox, and reopened on 2026-07-29 when that gate was removed (ADR-014). Accepted for a synthetic, non-public demonstration; ADR-010 records the design to restore before any real use	Build owner

Closed risks

#	Risk	How it closed
C-1	The private model lane could not be hosted anywhere available	Serverless GPU, deployed from the personal machine, reachable from both as an HTTPS endpoint
C-2	Cold starts exceeded the platform timeout, so the lane could never answer	Timeout stack raised in the right order, and later a smaller model chosen that loads in about a minute
C-3	Parallel agents woke a second GPU container and burned the budget	The lane is pinned to a single container and calls are serialised client-side
C-4	Anyone could sign up claiming any address and read that person's tickets	Closed 2026-07-28 by hard-gating sign-in on a verified inbox. REOPENED 2026-07-29 when that gate was removed by decision; it is now tracked as R-11 above
C-5	The verification gate locked out every seeded staff account	Fixed 2026-07-28, then made moot 2026-07-29 when the gate was removed
C-6	Approving an escalated ticket sent nothing to the customer	Fixed, and pinned by the first test in the suite
C-7	The repository had no automated tests at all	A gate-focused suite exists and runs free in about two seconds
C-8	Quality claims could not be defended, because no labelled evaluation set existed	30 labelled tickets plus <code>eval.py</code> , 2026-07-28. Classification accuracy and retrieval hit rate are now numbers, and the retrieval half re-runs for free

16. Traceability Matrix

Version 1, 2026-07-28. Each requirement from [01-requirements.md](#), the code that satisfies it, and the test that proves it. Verdicts are MET only where real code does the work.

Legend: **MET** implemented; **PARTIAL** intent covered but a stated piece is missing or done differently; **NOT MET** nothing implements it.

Functional

Req	Verdict	Code	Test
FR-1 form intake	MET	<code>frontend/src/app/new/page.tsx</code> , <code>POST /tickets</code> , <code>app/intake.py</code>	manual, live probe
FR-1 email intake	MET	<code>POST /email/sync</code> , <code>app/email_channel.py</code>	none
FR-1 voice transcript intake	PARTIAL	Parser branch exists in <code>app/intake.py</code> , but no endpoint supplies a transcript field, so the format is unreachable	none
FR-1 one canonical schema	MET	<code>Ticket</code> in <code>app/state.py</code>	none
FR-1 adapter interface	MET	<code>app/adapters.py</code>	none
FR-2 labels	MET	<code>classify()</code> in <code>app/graph.py</code> , <code>classify_agent</code> in <code>app/agents.py</code>	none
FR-2 difficulty	MET	<code>score_difficulty()</code> in <code>app/graph.py</code>	none
FR-2 classification confidence	NOT MET	Neither path returns one; only the draft carries a confidence	n/a
FR-2 sensitivity flag with reasons	MET	<code>detect_sensitivity()</code> in <code>app/graph.py</code>	none
FR-3 pattern scan	MET	<code>app/pii.py</code> , including a card checksum	none
FR-3 model check for context	MET	Sensitivity prompt in <code>app/graph.py</code>	none
FR-3 decided before any third-party call	MET	<code>think_model()</code> in <code>app/router.py</code> pins unlaned reasoning to the private lane	missing, and it is the system's strongest claim
FR-4 two-by-two	MET	<code>intended_model()</code> in <code>app/router.py</code>	none

Req	Verdict	Code	Test
router			
FR-4 sensitive never reaches cloud	MET	<code>reply_model()</code> and <code>think_model()</code> have no private-to-cloud path	missing
FR-5 ranked retrieval with scores	MET	<code>search()</code> in <code>app/kb.py</code> , floor of 60	none
FR-5 corpus holds articles and past resolutions	MET	<code>kb_import.py</code> , <code>index_resolved()</code>	none
FR-6 draft on the routed model	MET	<code>generate()</code> in <code>app/graph.py</code>	none
FR-6 citations	PARTIAL	The prompt asks for the article title in prose; no structured citation field	none
FR-6 draft confidence	MET	Control line parsed in <code>generate()</code>	none
FR-7 compliance gate	MET	<code>review()</code> in <code>app/graph.py</code> plus <code>policy.md</code>	none
FR-7 one regeneration retry	MET	<code>after_review()</code> capped by <code>review_count</code>	none
FR-7 second failure escalates	MET	<code>decide()</code>	none
FR-8 escalation rule	PARTIAL	High priority and sensitivity raise the confidence bar rather than forcing escalation	none
FR-8 assignee from the roster	MET	<code>assign()</code> in <code>app/roster.py</code>	none
FR-8 auto-send actually sends	MET	<code>_auto_dispatch()</code> and <code>dispatch_reply()</code> in <code>api.py</code>	<code>test_approve_draft_sends_and_hands_ball_to_customer</code> covers the approve path
FR-9 approve, reject, edit	MET	<code>POST /tickets/{id}/approve</code>	reject
FR-10 hash chain per step	MET	<code>app/audit.py</code> , <code>audit</code> reducer in <code>app/state.py</code>	missing tamper test

Req	Verdict	Code	Test
FR-10 tampering detectable	MET	<code>verify()</code> in <code>app/audit.py</code>	missing
FR-10 chain persisted	MET	Whole state stored as JSON in <code>app/store.py</code>	none
FR-11 index-back on resolve	MET	<code>/resolve</code> to <code>learn_agent()</code> to <code>index_resolved()</code>	none

Non-functional

Req	Verdict	Evidence	Test
NFR-1 no cloud fallback for sensitive	PARTIAL	No leak is possible; an outage ends in an error status instead of a human escalation	missing
NFR-2 latency captured and shown	MET	<code>set_processing_seconds()</code> in <code>api.py</code> , dashboard tiles	none
NFR-3 scale	MET	Seeded corpus, single-server design	n/a
NFR-4 graceful failure	PARTIAL	Two attempts and a 180 second cap, visible error; no backoff, no fallback model	none
NFR-5 accumulate, do not overwrite	MET	State in <code>app/state.py</code>	none
NFR-6 reusable boundaries	MET	Proven by the two systems forked from this one	n/a
NFR-7 synthetic data, secrets outside the repo	MET	Seed scripts, ignored environment file, blank example file	n/a

Requirement metrics

Metric	Verdict	Evidence
Classification accuracy	MET	<code>data/eval_set.jsonl</code> (30 labelled tickets) scored by <code>eval.py</code> : category 86.7%, priority 73.3%, both 66.7%, all 3 critical tickets correct
Retrieval hit rate	MET	Same set: 100% at top five over 22 answerable tickets, 36% at rank one, mean rank 1.86. Free to re-run
Auto-resolved against escalated	MET	<code>store.metrics()</code> , <code>bench.py</code> , dashboard
Compliance pass rate	MET	<code>store.metrics()</code> , dashboard
Average latency per ticket	MET	<code>store.metrics()</code> , warm and cold split
Cost per ticket	PARTIAL	GPU cost estimated; cloud token cost not tracked

Test coverage summary

A test layer exists (nine tests, free, no model calls) and covers the gates: ownership, roles, the resolved lock, the reopen path, and what approve sends. The six tests named in the original plan for the rule functions, the parsers, agent contracts, the tamper check and the sensitive-never-cloud assertion **do not exist yet**. That is the honest state, and the two most valuable to add are the tamper test and the sensitive-never-cloud test, because they are the two claims a reviewer is most likely to challenge.

17. User Guide

Version 1, 2026-07-28. For the people using the system, not the people building it. Two audiences: the customer filing a request, and the support agent handling it.

Part one: the customer

Creating an account

Open the portal, choose to sign up, and enter an email address and a password. You are signed in straight away: there is no email to open and nothing to confirm.

Filing a request

Sign in, choose to make a new request, give it a short subject and describe the problem. Submit. The request appears immediately with a progress bar while the system works on it. You do not need to wait on the page.

What happens next

One of three things:

- **You get an answer.** The reply appears in your thread, usually within a minute.
- **You get a question.** If something is missing, an order number for example, the system asks for it. Reply in the thread and it picks up again.
- **A person takes over.** For anything sensitive, urgent, or that the system is not confident about, a member of staff is assigned and you are told so.

Asking for a human in your message always brings a person in, with no automated reply first.

Following up and closing

Your requests list shows everything you have filed and its state. Open one to read the thread and reply. When you are satisfied, mark it resolved and optionally leave a rating. If the answer turns out not to have worked, a resolved request can be reopened from the same screen.

Part two: the support agent

The queue

Signing in as staff opens the workspace. The queue lists live tickets with their status, category, priority, tags, assignee and age. Filter by status, category or tag, or search the text. A row showing a moving bar is still being processed.

Administrators can also switch the scope to the archive; staff cannot, and instead see one customer's past tickets from inside that customer's live ticket.

Reviewing a ticket

Opening a ticket gives you three columns.

On the left, who you are talking to. Their plan and account standing, their recent orders with status and tracking, their recent charges, the ticket's tags, and their past tickets. These are the same lookups the agents make while drafting, put on screen so **you are never worse informed than the system that wrote the reply**. A sender who is not in the customer records says so plainly, which is normal for tickets arriving by email or from Jira.

In the middle, the conversation, laid out like a messaging thread: the customer on the left, your replies on the right, and internal notes as a line between them that the customer never sees. While the pipeline is still working, a typing indicator shows it is mid-flight rather than stuck.

On the right, the verdict and your reply. The system states what it decided, how confident it was and on what grounds. Its proposed wording appears as a **suggested reply** above an empty box: you can use it as it stands, load it into the box to edit, or dismiss it and write your own. Nothing is ever pre-loaded into the box on your behalf, and nothing sends without you pressing send.

Three actions:

Action	Effect
Use this	Accepts the suggested wording and sends it, through the channel the ticket arrived on, handing the conversation back to the customer
Use and edit	Loads the suggestion into the box so you can change it before sending. Nothing is sent until you press Send
Dismiss	Puts the suggestion away and leaves you an empty box to write your own reply
Send	Sends whatever is in the box. If you changed the wording, your version is saved over the draft first, so what is recorded is what was actually sent. If a reply had already gone out automatically, your edit corrects the thread record; the copy already delivered by email cannot be recalled
Reject	Marks the draft unusable. Nothing is sent
Mark resolved	Closes the ticket and files it in the archive

You can also add an internal note, which the customer never sees, and add or remove tags.

Related tickets

Two different jobs sit in the side panel, and they are not the same thing:

Action	Effect
Link	A cross-reference. Both tickets stay open and stay separate. Useful when two customers report the same outage
Fold in	Folds another ticket into this one and resolves that other ticket. The button arms on the first press and fires on the second, because it closes something

Existing relations show as chips: ➡ for a link, ◀ for a ticket that was folded in. If this ticket was itself folded into another, the panel says so and links you there. A ticket already merged once cannot be merged again.

Attachments

The side panel lists every file on the ticket with its size, and each name downloads it. **Attach a file** accepts images, PDF, plain text and CSV up to 5 MB; anything else is refused with the reason stated on screen. A resolved ticket is locked, so the upload button disappears.

Templates, used as macros

The templates library holds canned replies. On a ticket, the saved replies appear as a row of macro chips above the draft: click one and it replaces the draft, which you can then edit before sending. The chip stays marked so you can see which macro is in the box.

Only administrators can create, change or delete templates, and **authoring is still API-only**: there is no templates management screen, so the library is maintained through the API documentation page. Applying them is fully on screen.

Bringing work in from elsewhere

Two buttons pull work into the queue: one syncs the mailbox and turns unread messages into tickets, ignoring automated mail, and the other pulls new issues from the connected project tracker. Replies go back out through whichever channel the ticket came from.

Resolving

Resolving archives the ticket, closes the linked tracker issue if there is one, and files the resolution into the knowledge base so future tickets can retrieve it. A resolved ticket is locked: no more approvals, edits, notes or tags. Reopen it if it turns out not to be finished.

The metrics page

Shows volume, how much was auto-resolved against escalated, the compliance pass rate, average handling time split into warm and cold runs, an estimated cost per ticket, average customer rating, and average time to resolution. It recalculates from live data every time you open it.

Part three: things worth knowing

- **The first ticket after a quiet period is slow.** The private model server sleeps and takes about a minute to wake. This is why a demonstration starts with a throwaway ticket ten minutes early.
- **Sensitive tickets are handled differently on purpose.** Anything containing personal, financial, medical or legal detail is processed only on the private model, never by an outside provider, and the ticket says so.
- **The system never invents policy.** If the knowledge base does not cover a question, it is meant to ask or escalate rather than guess.
- **Everything is recorded.** Every step is written into a chain that shows if any past entry is altered.

18. Release Notes

Version 1, 2026-07-29. Grouped by theme rather than by tag: this project has no version tags, first commit 2026-07-05. Full detail is in the commit history.

2026-07-29 later, email verification removed

This reverses the hard gate shipped the day before, by decision (ADR-014), and it gives something up.

- Signing up now signs you straight in. There is no link to open, no check-your-inbox screen, no resend, and no verify page.
- Removed: the verification emailer, `/auth/verify` and `/auth/request-verify-token`, `requires_verification` on the login router, the `_require_verified` check on the three customer paths, and `FRONTEND_URL`, which existed only to build the link and is now dead configuration.
- **What this costs, stated plainly:** open signup means anyone can register an address they do not own, and because a customer's tickets are matched by their account address, they would see tickets filed from it. That is accepted for a synthetic demonstration system with no public deployment. It is tracked as R-11 in the risk register and named as the largest open hole in the security review, and ADR-010 records how the gate was built if it needs restoring.
- The auth library's `is_verified` column stays but is now inert; dropping a library-owned column would need a migration for nothing. The suite went from 10 tests to 9.

2026-07-29, the API-only features reach the screen

Four capabilities were already built, tested and reachable at the API, and invisible in the product. This release is the wiring, not new machinery: **no endpoint changed**, so every benchmark and test result carries over unaltered.

- **Macros.** Saved replies appear as chips above the draft on a ticket. Clicking one applies it through the existing endpoint and shows the body the server actually stored, rather than a local guess at it.
- **Merge and link.** The side panel now distinguishes the two: a link is a cross-reference that leaves both tickets open, a fold-in closes the other ticket and therefore arms before it fires. Existing relations render as chips in both directions, including the case where this ticket was the one folded away.
- **Attachments.** Upload with the type and size ceiling stated on screen, and the server's own refusal message shown when it rejects one. Files download straight from the API, whose session cookie is scoped to the host rather than the port.
- **The last three pages joined the component layer:** the customer landing page, the email verification page and the performance dashboard. The dashboard was the furthest from the house style and gained the most. (The verification page was deleted hours later by the change above, so only two of those three survive.)

Both remaining honest gaps are recorded under known issues: template authoring has no screen, and the file picker sits on the staff side only.

2026-07-28 later, Ledger refined

- **The interface gained a component layer.** The styling already had a deliberate printed-paper identity and a full token set, but no named components, so every button and input was a hand-typed string of utility classes retyped across ten pages, and the pages had begun to drift apart. `globals.css` now defines the controls once (`.btn` and its variants, `.card`, `.panel`, `.input`, `.field`, `.badge`, `.chip`, `.row`), referencing only the existing tokens and the three font variables, and the five core pages (staff queue, staff ticket detail, customer thread, login, new request) were rewritten onto them. The remaining three followed on 2026-07-29, so **every page in the product now draws from the component layer**. No endpoint changed, so the tests and both benchmark results stay valid by construction.

- **A font swap is now a one-file job:** the components name only the font variables, so replacing the three families means dropping new files into the fonts folder and editing `fonts.ts`.
- **Two dead pre-auth pages deleted.** One rendered the company metrics page to any visitor without a session (the API behind it did require one); the other duplicated the real staff ticket page. Neither was linked from anywhere.
- **The queue now shows the assignee.** The backend always computed it; only the detail page displayed it.
- Two literal colours that bypassed the token set were removed along the way.

2026-07-28, verification hard gate

- Sign-in now requires a verified email address. An unverified attempt is refused with its own distinct reason, so the interface can tell it apart from a wrong password, and no session is issued.
- Signing up no longer signs you in. The screen asks you to open the link, with a resend option that cannot be used to discover which addresses exist.
- **Breaking, and caught by testing:** the gate initially locked out every seeded staff and administrator account, because the seeder only marked the verified flag on accounts it created. The seeder now repairs existing accounts. Re-run it after pulling this change.
- A blank `FRONTEND_URL` now falls back to the local address instead of producing a link with no host.

2026-07-27, lifecycle, access control and the first tests

- Reopen a resolved ticket from either side, which renames it back to a live identifier and reconnects its tracker link.
- Archive browsing became administrator-only, with staff instead getting need-to-know access to one customer's history from inside a live ticket.
- Progress bars on rows and detail screens while a ticket is being processed.
- Email verification built: link sending, a verification page, and a check-your-inbox state.
- **The repository got its first automated tests**, ten of them, free and free of model calls. Fixed a real defect on the way: the API needed a live database merely to import, which made testing impossible.
- Performance dashboard with latency and estimated cost.

2026-07-21, resolve semantics and real auto-send

- Resolving became atomic and safe against a race with an in-flight pipeline run, so a resolved ticket can no longer be resurrected.
- Auto-send genuinely sends, immediately, through the channel the ticket arrived on, rather than waiting for a human click.
- A resolved ticket is locked against further action, returning a conflict rather than silently accepting it.
- Queue gained scope filtering, a status vocabulary, and tag and creation columns.
- Timestamps stamped as aware UTC; the previous naive local time skewed sort order and the service clocks by hours.

2026-07-18 to 2026-07-19, accounts and the interface

- Accounts, roles and cookie sessions, with ownership walls so a customer reaches only their own tickets.
- The full web interface: queue, ticket detail with the system's reasoning panel, customer portal, thread view, ratings.
- Project-tracker channel with reply-back and status sync.

2026-07-16 to 2026-07-17, agent quality and cost

- Twelve-tool universe for the autonomous agents, plus a coherent synthetic world of customers, orders and charges for those tools to read.

- Confidence bar recalibrated from 90 to 85 after correct answers scoring 87 were being escalated; escalation fell from a third of the batch to roughly one ticket in twelve.
- Tool-loop deduplication and a forced finish, so an agent cannot spin.
- Mailbox channel with a whole-run time cap, and one retry on timeout to absorb a cold model container.

2026-07-13 to 2026-07-14, from single answers to conversations

- Multi-turn threads with a lifecycle, replies that re-run the pipeline over the whole conversation, and knowledge write-back moved to the resolve action so only confirmed answers are learned.
- Helpdesk features: tags, search and filters, internal notes, service timers, ratings, templates, merge and link, attachments.

2026-07-12 and earlier, the skeleton

- The fixed pipeline with its ten nodes, the two-by-two model router, the private and cloud lanes, the knowledge base with a seeded corpus, the hash-chain audit trail, and the mode switch that keeps both the fixed and the autonomous pipelines available.

Known issues carried forward

- The transcript intake format has a parser but no endpoint to reach it.
- A private-lane outage ends in an error status rather than an escalation to a person.
- Cloud token cost is not tracked, so full-tier runs under-report cost.
- Retrieval finds the right article in the top five 100 percent of the time but puts it first only 36 percent of the time.
- Templates can be applied from the workspace but not authored there: creating, editing and deleting a canned reply is still API-only.
- Attachments are staff-only. All three attachment endpoints require a staff session, so a customer cannot add a screenshot from the portal; an agent has to attach it on their behalf.

19. Handover

Version 1, 2026-07-28. What the next owner needs, in the order they will need it.

1. Get it running

Follow the repository `README.md` for first-time installation, then [08-runbook.md](#) for daily operation. You need Docker, the Python runner `uv`, Node, and an environment file built from `.env.example`. The two model endpoints and their shared token are the only values you cannot invent locally; without them the API refuses to start.

Expect this order: start the containers, run the four seed scripts, start the API, start the web app, sign in with a seeded staff account.

2. Read these three first

1. [02-hld.md](#) for the shape of the system.
2. [03-llid.md](#) section 3 and 4, the pipeline and the decision rule, which is where all the judgement lives.
3. [06-adr-log.md](#) for why things are the way they are, especially the two-pipeline decision and the privacy lane.

3. The things that will surprise you

1. **There are two pipelines**, chosen by an environment variable, and any change to pipeline internals has to be made in both in the same sitting. Forgetting this is how a feature has previously shipped in one mode only.
2. **The two pipelines disagree about capitalisation**. The fixed one lower-cases every classification field; the autonomous one returns the model's raw wording. Any code reading a classification must lower-case first, or it will work in exactly one mode.
3. **The model lane sleeps**. The first call after an idle period takes about a minute, and it costs real money to wake. Batch anything paid into one warm window.
4. **The whole pipeline state is stored as one JSON column**. Adding a field to the state needs no migration; the queue's filter columns are the only normalised ones.
5. **Corporate security tooling on a Windows machine interferes twice**: a local proxy hijacks the vector database's gRPC port unless localhost is excluded, and a content filter rewrites credential-shaped strings, so configuration files should be checked after any bulk edit.
6. **All three sibling systems run on :8000 and :3000**, and every front end hardcodes `http://localhost:8000`. Run one app at a time; the database containers have distinct ports and can all stay up.
7. **Styling goes through the component layer in `globals.css`**. New interface work should use the named components (`.btn`, `.card`, `.input`, `.badge` and friends), never a literal colour or font family; that is what keeps a font swap a one-file edit in `fonts.ts`.

4. Where the important logic lives

Question	File
Which model handles this ticket	<code>app/router.py</code>
Send automatically or ask a human	<code>decide()</code> in <code>app/graph.py</code> , with the bar in <code>app/state.py</code>
What counts as sensitive	<code>detect_sensitivity()</code> in <code>app/graph.py</code> and <code>app/pii.py</code>
What the compliance gate enforces	<code>review()</code> in <code>app/graph.py</code> and <code>policy.md</code>
What happens after a run finishes	<code>_process()</code> and <code>_auto_dispatch()</code> in <code>api.py</code>
Everything touching the database	<code>app/store.py</code>

5. Open work, in the order worth doing

1. **Retrieval at rank one**, 36 percent against 100 percent at rank five. The right article is nearly always in the list but rarely first, and the drafting prompt reads the top hits, so this is the next measurable quality win. Re-running the retrieval half of `eval.py` is free, which makes it cheap to iterate on. (Priority classification was the previous entry here and has been done: 66.7 to 73.3 percent, all three critical tickets fixed, by rewriting the priority definitions in both pipelines.)
2. **The two missing safety tests**: one proving the audit chain detects tampering, one asserting a sensitive ticket never reaches the cloud lane.
3. **Escalate rather than error when the private lane is down**. Today a lane outage ends in an error status, which meets the privacy rule but not the intent.
4. **An endpoint that accepts a transcript**, so the third intake format is reachable.
5. **Track cloud token cost**, so full-tier runs report their true cost.
6. **Extract the shared core**, per the decision recorded in the planning repository. Deliberately scheduled after the current deadline.

6. Operational cautions

- Never run `docker compose down -v` unless you intend to lose the data and re-seed.
- Changing the authentication secret signs everyone out.
- The database password in the environment file and the one in the compose file must agree; Postgres only applies credentials when it first initialises an empty volume.
- Seeded development credentials are in the `README.md` and must be rotated before the system is reachable by anyone else.

7. Related repositories

Two sibling systems were built from this skeleton: a legal contract review system and an enterprise AI governance system. They share several modules by copy, which is measured and planned for in the core-sharing decision in the planning repository. A fix made in a shared-looking module here is worth checking in both of them.